

Tháng 3 2026

DECORATOR

Wrapper

Trình bày bởi Nhóm 5

NỘI DUNG

01. Tổng quan

02. Vấn đề, Giải pháp

03. Cấu trúc

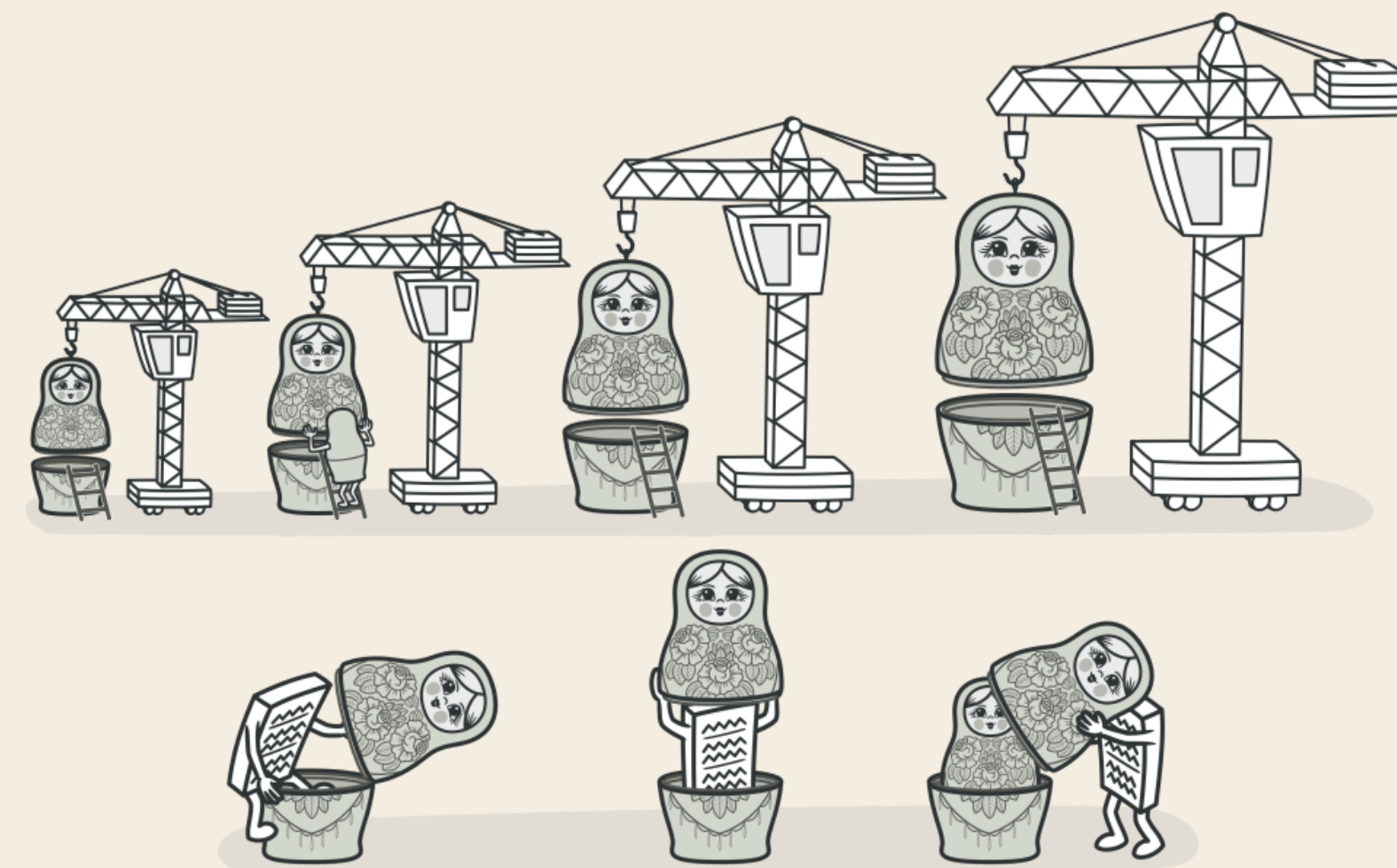
04. Cách ứng dụng

05. Ưu nhược điểm

06. Các mẫu liên quan

2 TỔNG QUAN

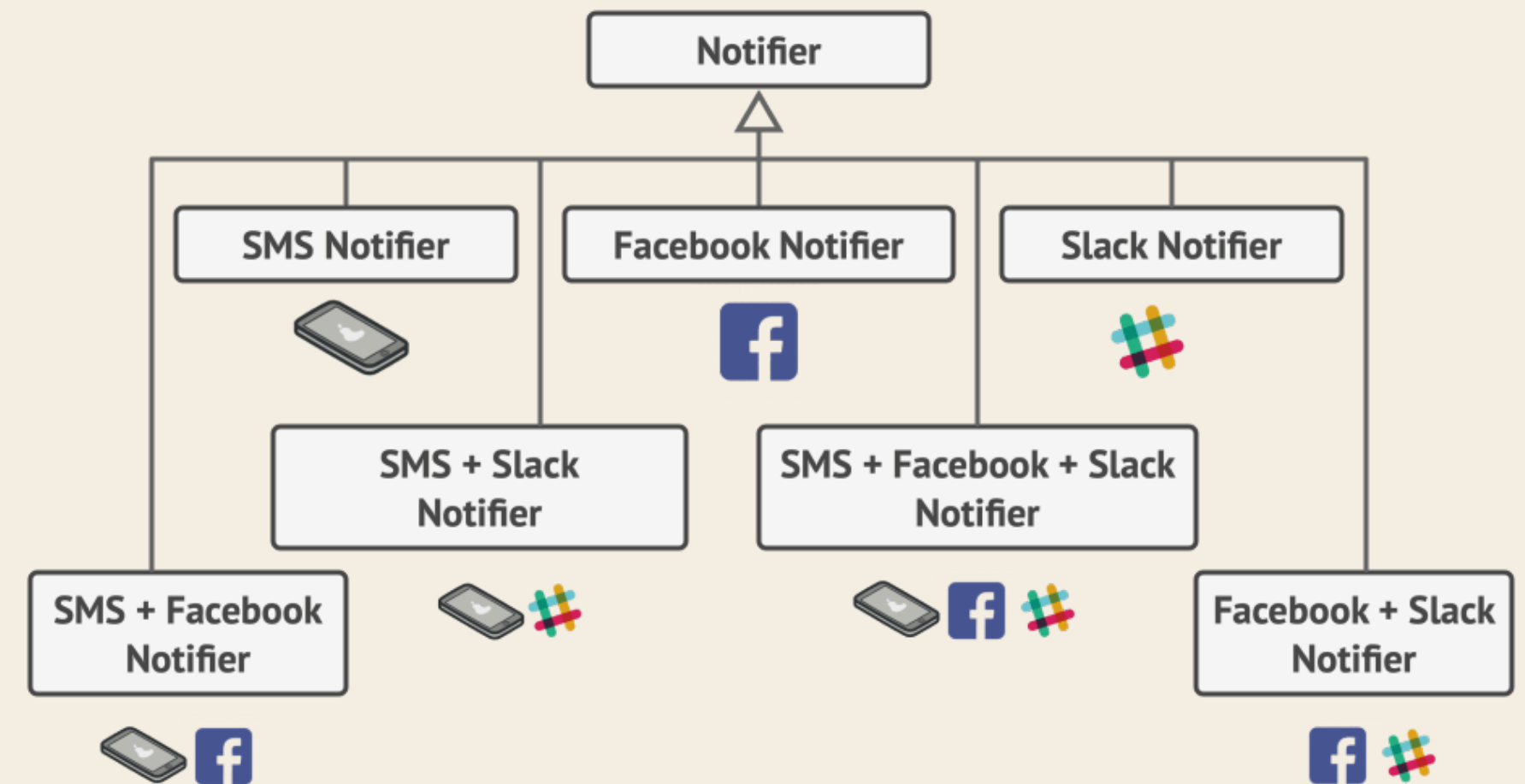
- Tên: Decorator, Wrapper
- Loại: Structural Design, Mẫu cấu trúc
- Mục đích: thêm tính năng cho các lớp bằng cách bao bọc bởi lớp chứa tính năng đó.



Hình ảnh minh họa khái niệm Lớp bao bọc
bởi Refactoring.Guru

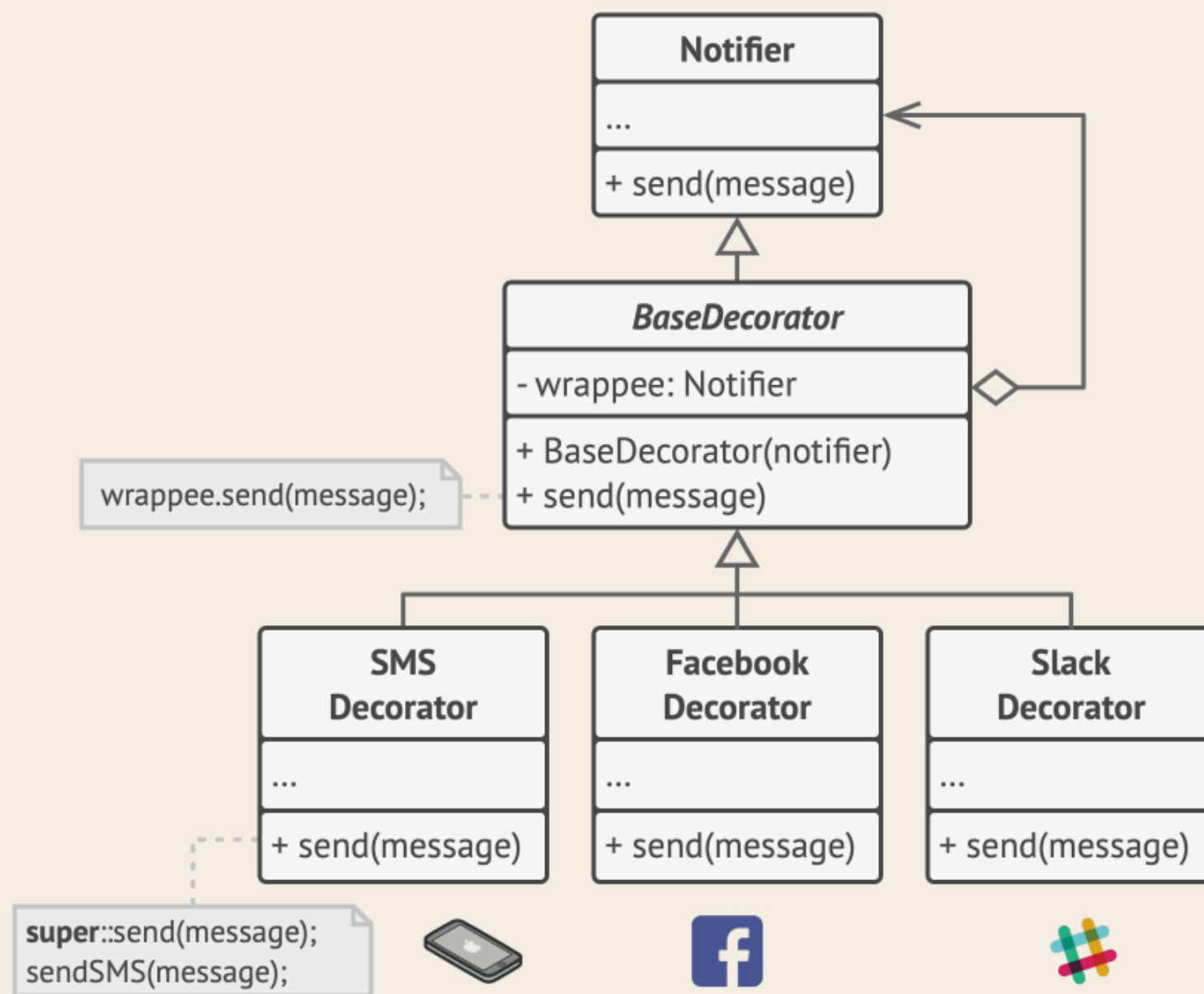
VẤN ĐỀ

- Notifier: gửi thông báo email
- Kế thừa: gửi thông báo tin nhắn, Facebook, và Slack.
- Mỗi người chỉ được sử dụng một lớp.
- Không thể kế thừa lớp mới khi chương trình đang chạy, phải tạo trước.

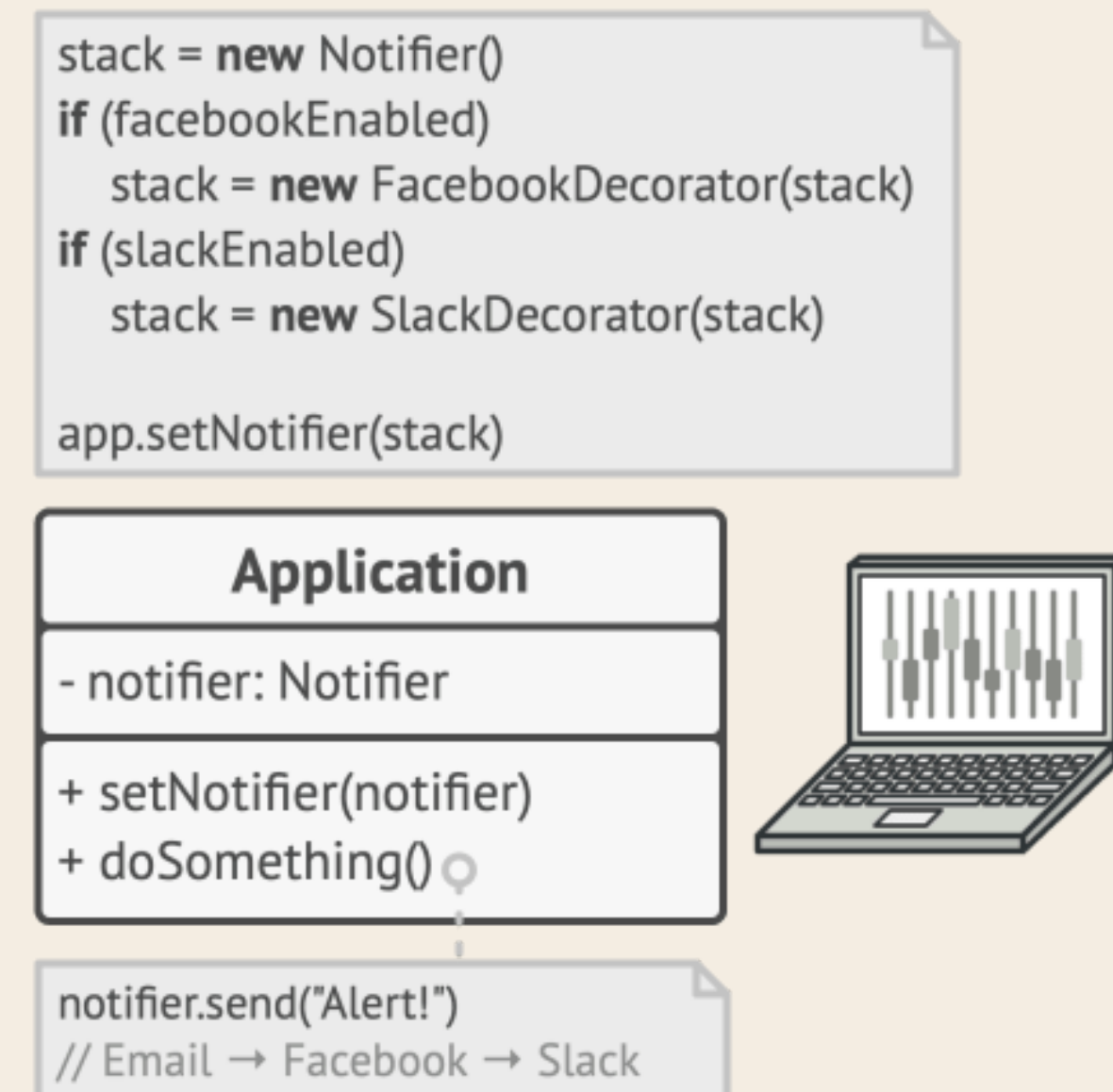
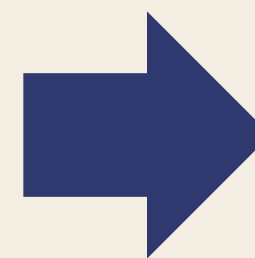


Hình ảnh minh họa khái niệm kế thừa một lớp
bởi Refactoring.Guru

GIẢI PHÁP



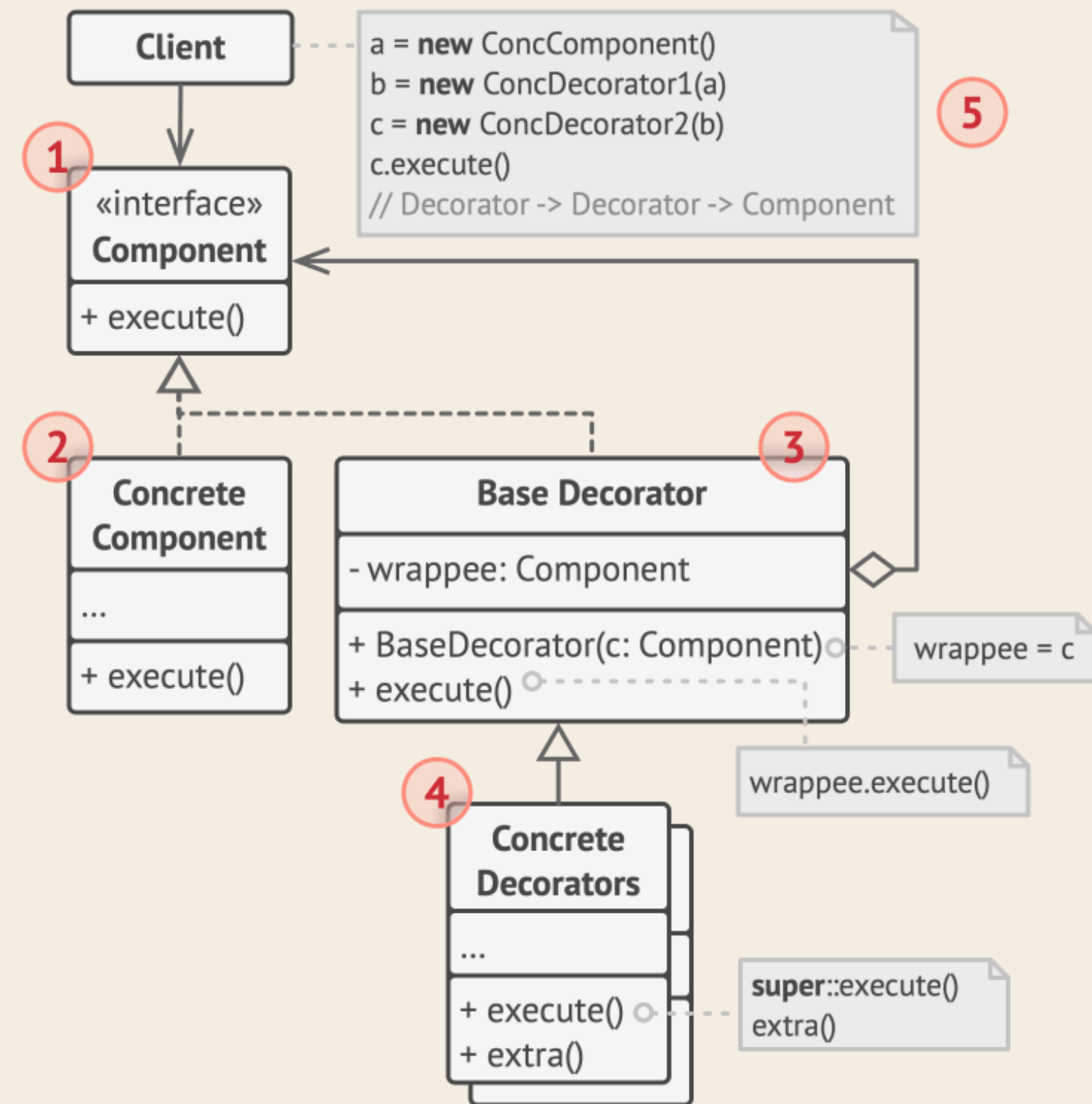
Hình ảnh minh họa việc kế thừa từ một lớp cơ bản bởi Refactoring.Guru



Hình ảnh minh họa việc kết hợp các lớp vào một đối tượng bởi Refactoring.Guru

CẤU TRÚC

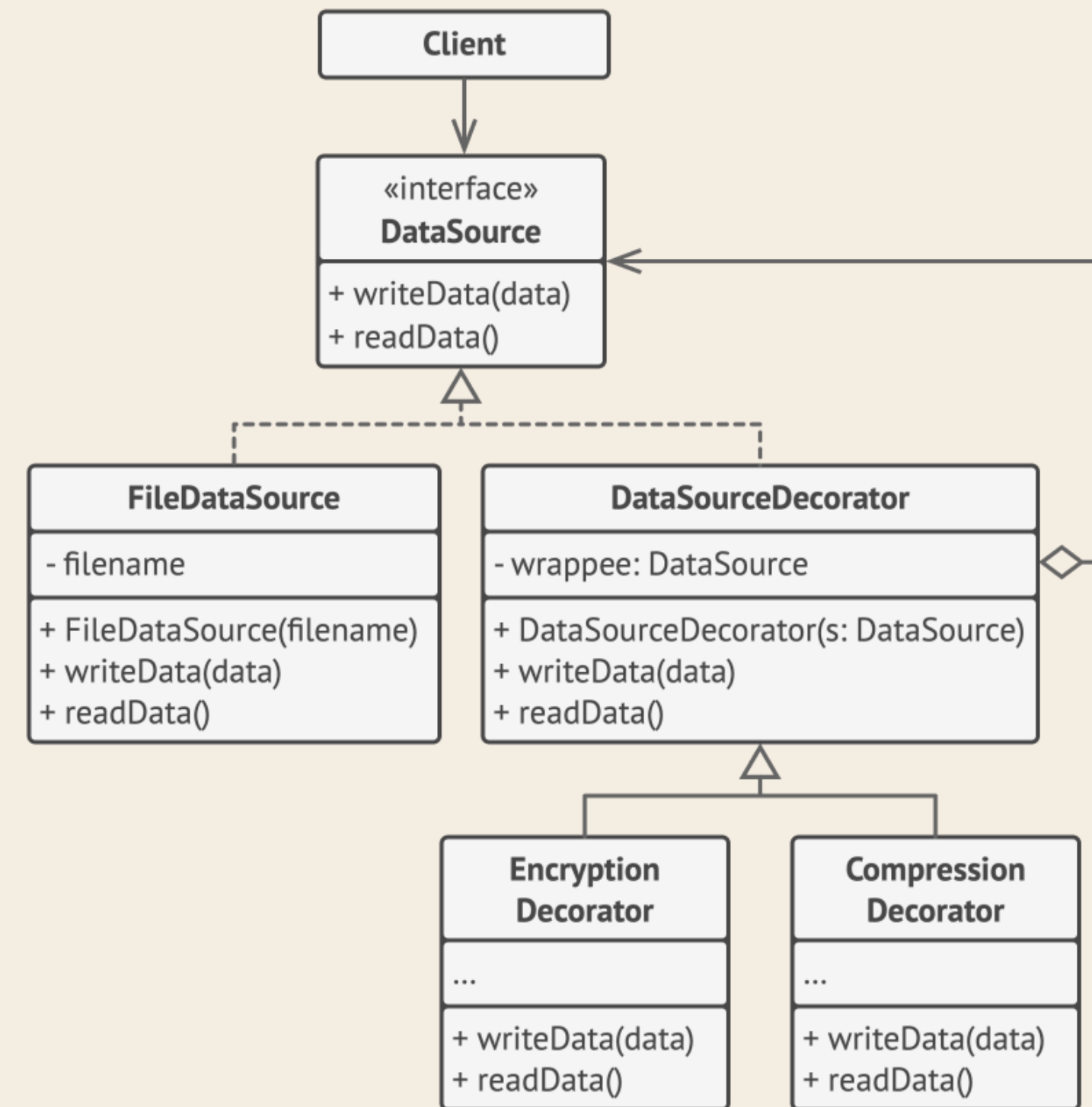
1. Component: lớp trừu tượng cho các Wrapper và các lớp bị Wrapper bao bọc.
2. Concrete Component: lớp bị Wrapper bao bọc.
3. Base Decorator: lớp trừu tượng cho các Wrapper.
4. Concrete Decorator: Wrapper.
5. Client: code thao tác với các lớp kế thừa.



Hình ảnh minh họa cấu trúc
mẫu thiết kế Decorator
bởi Refactoring.Guru

CÁCH ỨNG DỤNG

- Cần thêm tính năng cho đối tượng khi chương trình chạy
- Tránh việc sử dụng kế thừa
 - Bị sử dụng vô tội vạ
 - Lớp cha dễ ảnh hưởng lớp con



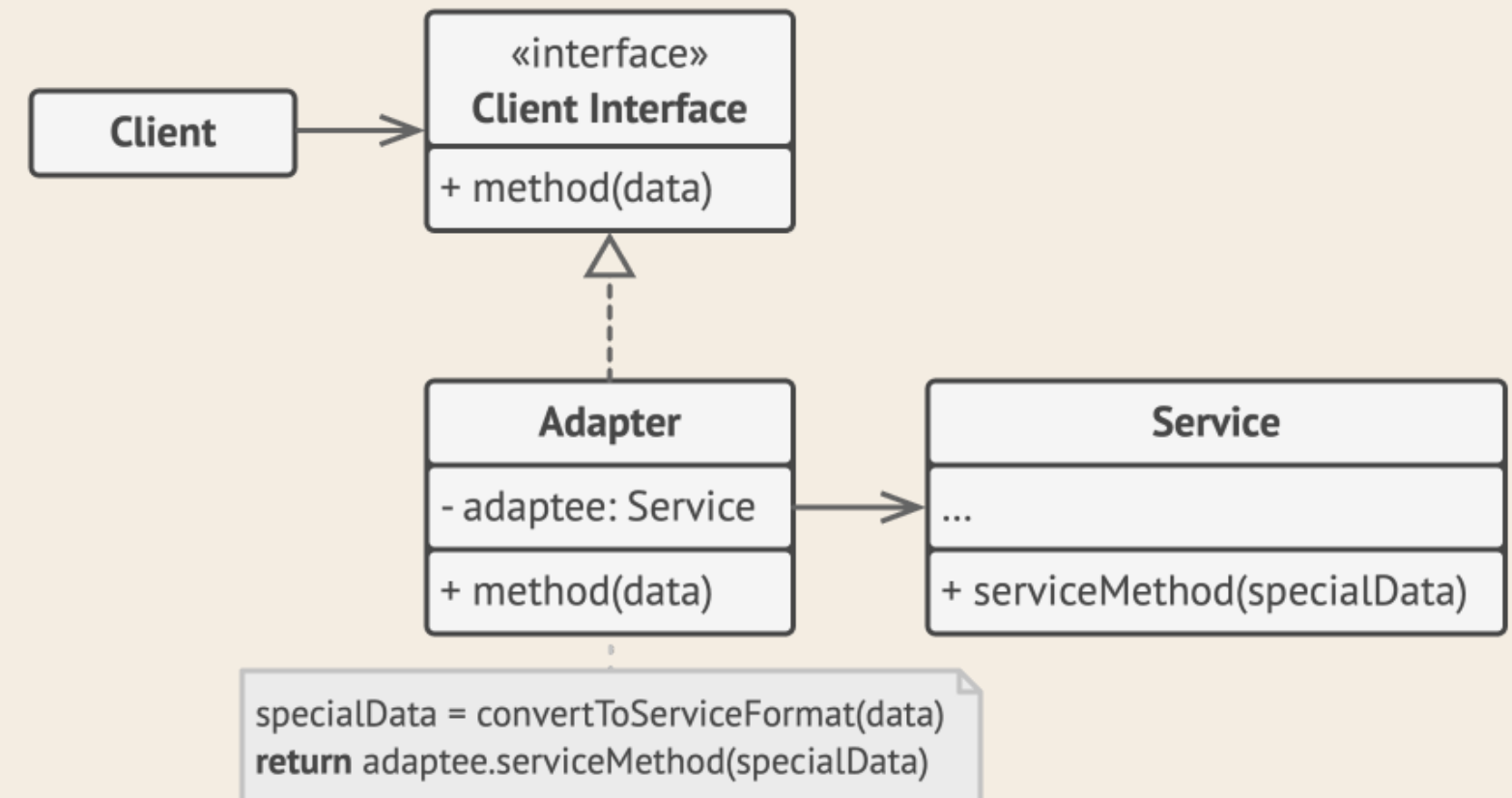
Hình ảnh minh họa cách ứng dụng
vào việc mã hoá và nén tệp tin
bởi Refactoring.Guru

2 ƯU NHƯỢC ĐIỂM

Ưu điểm	Nhược điểm
Single-responsibility Principle: tách các tính năng ra thành các lớp riêng	Việc thêm/bỏ lớp hoạt động như Stack
Thêm tính năng mà không cần kế thừa	
Thêm/bỏ tính năng trong lúc chương trình chạy	

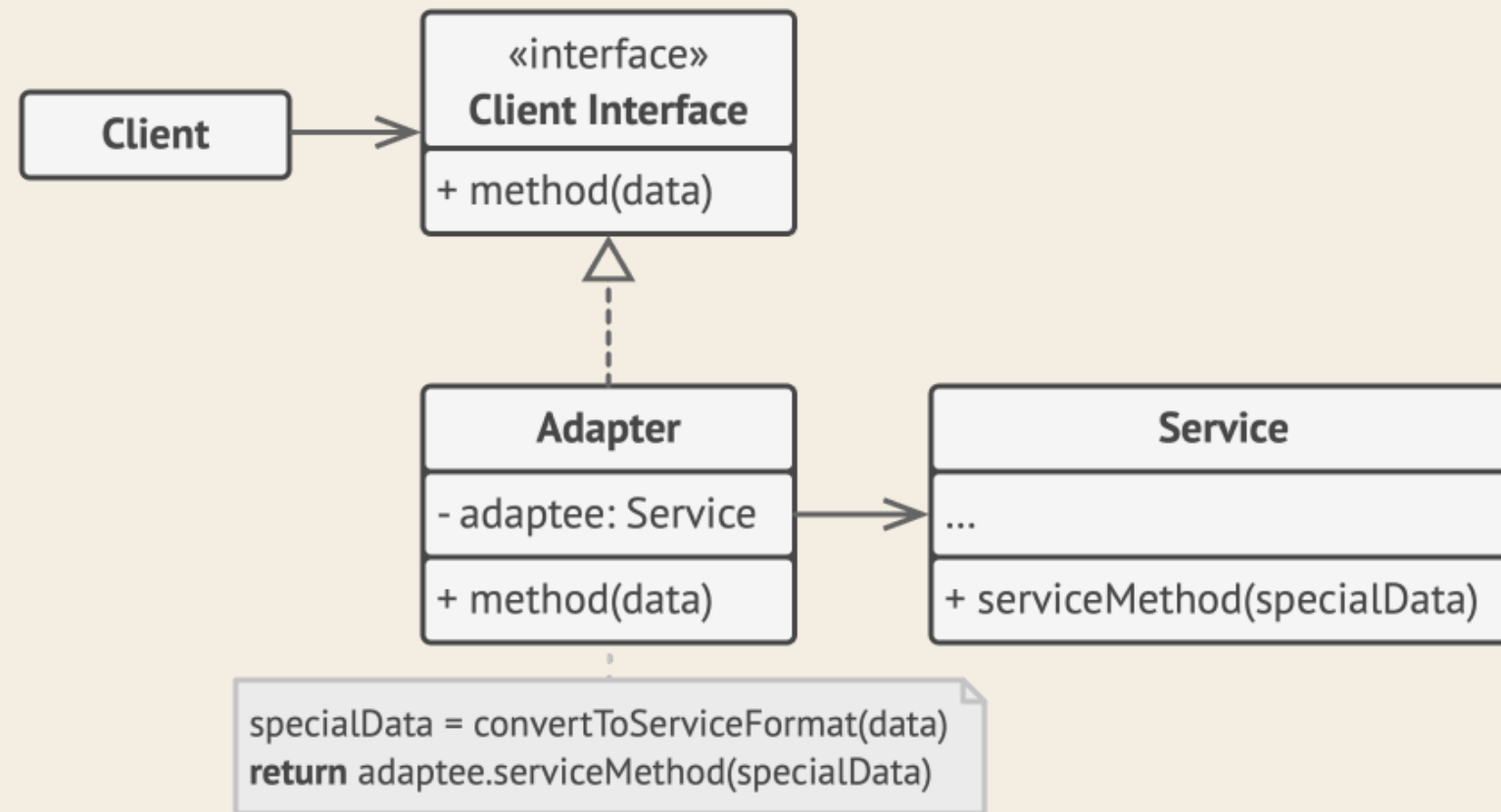
CÁC MẪU LIÊN QUAN

- Adapter cũng là Wrapper
 - Adapter: sửa tên phương thức
 - Decorator: giữ nguyên / thêm các phương thức

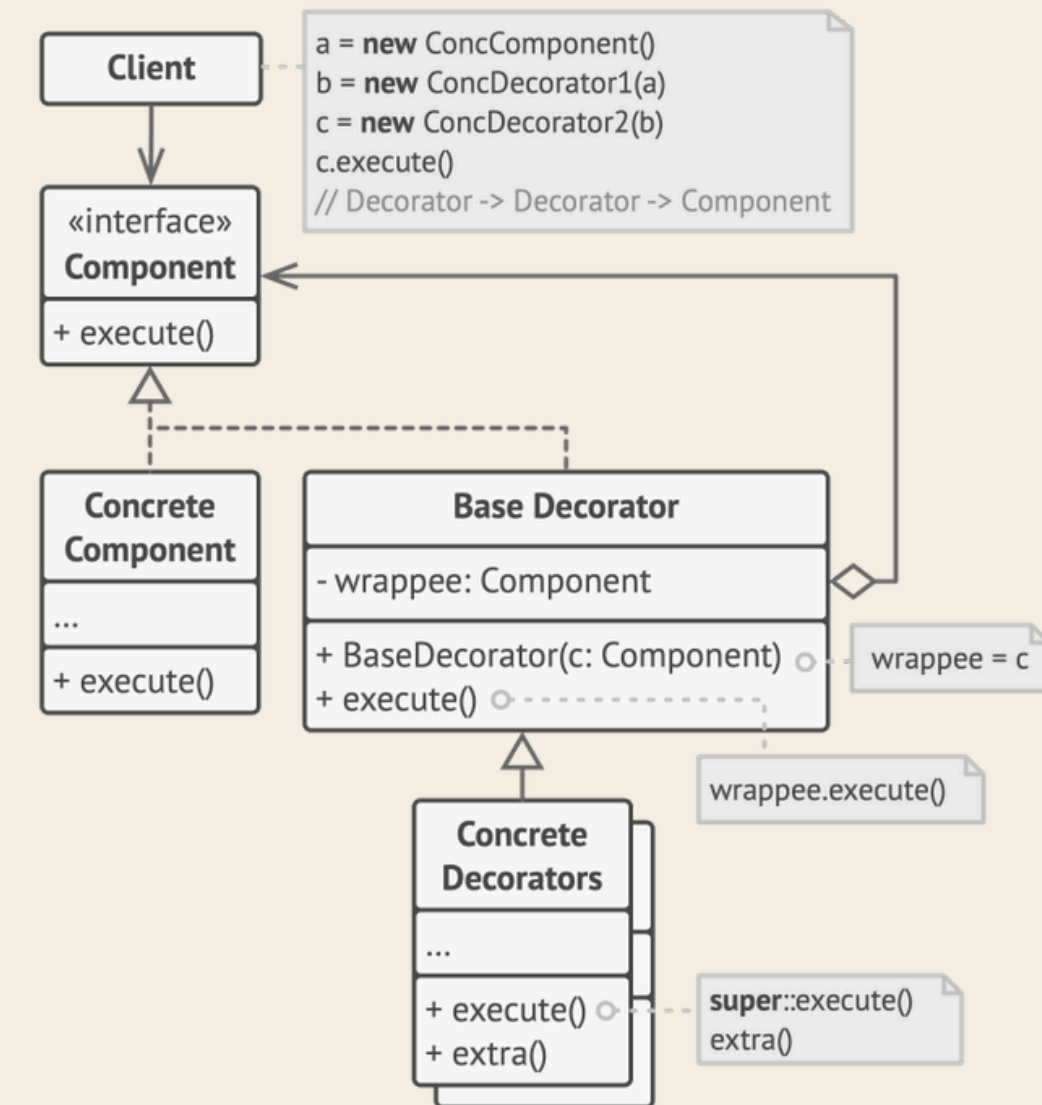


Hình ảnh minh họa cấu trúc
mẫu thiết kế Adapter
bởi Refactoring.Guru

CÁC MẪU LIÊN QUAN



Hình ảnh minh họa cấu trúc
mẫu thiết kế Adapter
bởi Refactoring.Guru



Hình ảnh minh họa cấu trúc
mẫu thiết kế Decorator
bởi Refactoring.Guru

CODE MINH HỌA

Interfaces/ITool.cs

```
namespace DecoratorDemo.Interfaces
{
    internal interface ITool
    {
        double Durability { get; }
    }
}
```

Interfaces/ISword.cs

```
namespace DecoratorDemo.Interfaces
{
    internal interface ISword : ITool
    {
        double AttackDamage { get; }
        double AttackSpeed { get; }
    }
}
```

Interfaces/SwordDecorator.cs

```
namespace DecoratorDemo.Interfaces
{
    internal abstract class SwordDecorator : ISword
    {
        public ISword _innerSword;

        public SwordDecorator(ISword innerSword)
        {
            _innerSword = innerSword;
        }

        public virtual double AttackDamage => _innerSword.AttackDamage;
        public virtual double AttackSpeed => _innerSword.AttackSpeed;
        public virtual double Durability => _innerSword.Durability;
    }
}
```

Implements/DiamondSword.cs

```
namespace DecoratorDemo.Implements
{
    internal class DiamondSword : ISword
    {
        public double AttackDamage => 7;
        public double AttackSpeed => 1.6;
        public double Durability => 1561;
        public double PercentDrop => 100;
    }
}
```

CODE MINH HỌA

Implements/DecoratorConcretes/Sharpness_V.cs

```
namespace DecoratorDemo.Implements.DecoratorConcretes
{
    internal class Sharpness_V : SwordDecorator
    {
        public Sharpness_V(ISword innerSword) : base(innerSword) { }

        public override double AttackDamage => base.AttackDamage + 3;
    }
}
```

Implements/DecoratorConcretes/Unbreaking_III.cs

```
namespace DecoratorDemo.Implements.DecoratorConcretes
{
    internal class Unbreaking_III : SwordDecorator
    {
        public Unbreaking_III(ISword innerSword) : base(innerSword) { }

        public override double Durability => base.Durability * (1 + 3d/7);
    }
}
```

Implements/DecoratorConcretes/FireAspect_II.cs

```
namespace DecoratorDemo.Implements.DecoratorConcretes
{
    internal class FireAspect_II : SwordDecorator
    {
        public FireAspect_II(ISword innerSword) : base(innerSword) { }

        public double FireDamage => 1;
    }
}
```

Program.cs

```
using DecoratorDemo.Implements;
using DecoratorDemo.Implements.DecoratorConcretes;
using DecoratorDemo.Interfaces;

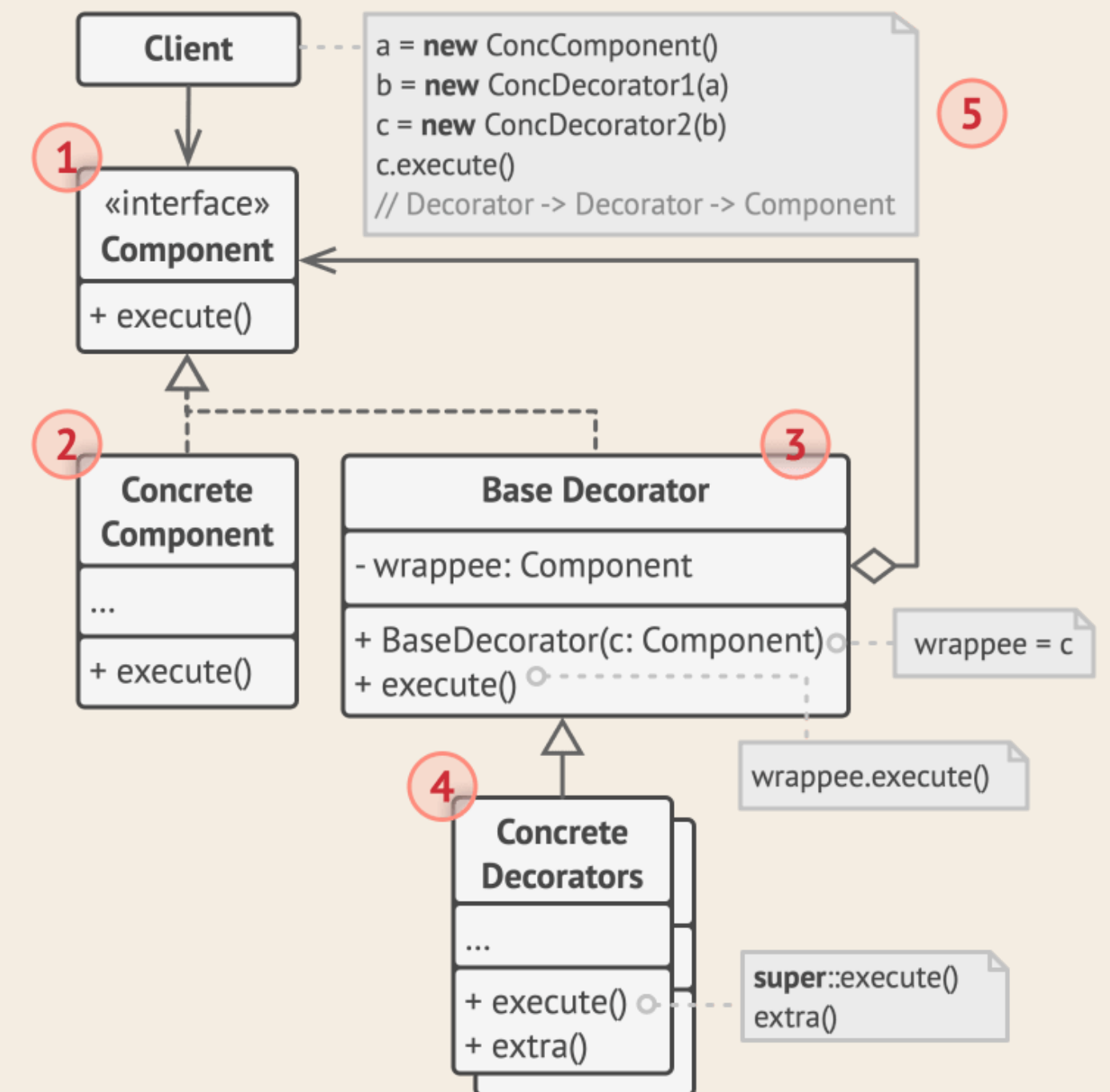
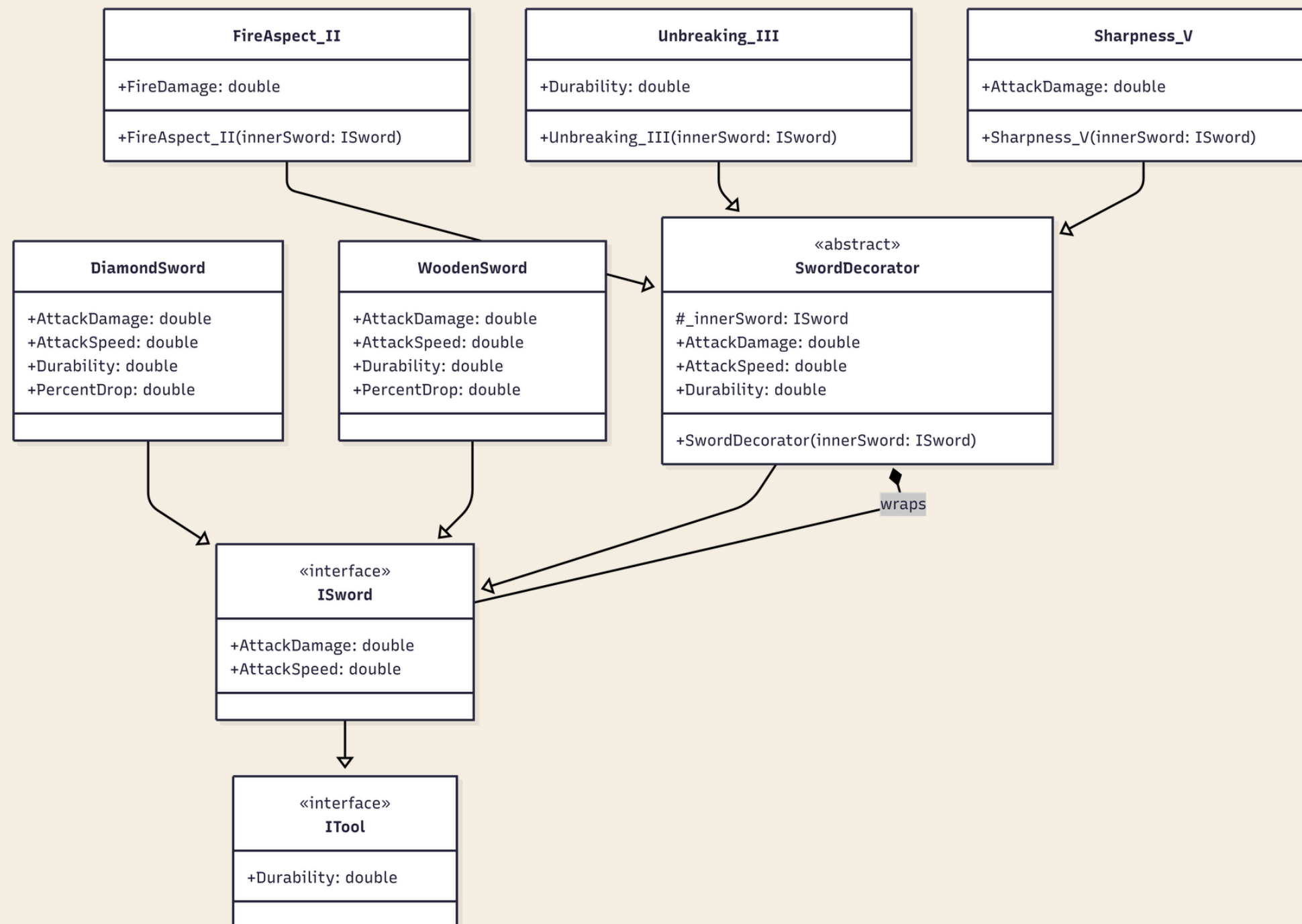
ISword diamondSword = new DiamondSword();
PrintInformation(diamondSword);

diamondSword = new Unbreaking_III(diamondSword);
PrintInformation(diamondSword);

diamondSword = new Sharpness_V(diamondSword);
PrintInformation(diamondSword);

void PrintInformation(ISword sword)
{
    Console.WriteLine($"""
        DiamondSword information:
            AttackDamage: {diamondSword.AttackDamage}
            AttackSpeed: {diamondSword.AttackSpeed}
            Durability: {diamondSword.Durability}
        """);
}
```

CLASS DIAGRAM



THÀNH VIÊN

Trương Hoàng Phúc

Trình bày
Decorator

Lê Minh Phát

Mã nguồn minh hoạ

Ya Đạt

Trình bày
Bridge

Tháng 3 2026

THANK YOU

for Listening

Trình bày bởi Nhóm 5